

# Rekurrenser i algoritmer — sammenfatningsark

Dette ark giver et hurtigt overblik over, hvordan vi opstiller og løser rekurrensligninger for almindelige algoritmer (især divide-and-conquer). Brug det som tjekliste til eksamen.

## 1 Hvad er en rekurrens?

En rekurrens beskriver køretiden for et problem ved at udtrykke den via tiden for mindre delproblemer.

### Idé:

Tiden for  $n$  afhænger af tiden for et mindre input + arbejdet udenfor rekursionen.

Eksempel:

$$T(n) = T(n/2) + n$$

## 2 Standardformer (meget vigtige)

| Rekurrens            | Intuition                         | Løsning (Big-O) |
|----------------------|-----------------------------------|-----------------|
| $T(n) = T(n-1) + c$  | én mere operation pr. trin        | $O(n)$          |
| $T(n) = T(n-1) + n$  | $1 + 2 + \dots + n$               | $O(n^2)$        |
| $T(n) = T(n/2) + c$  | halvering                         | $O(\log n)$     |
| $T(n) = T(n/2) + n$  | halvering + lineært arbejde       | $O(n \log n)$   |
| $T(n) = 2T(n/2) + n$ | to halvdele + merge               | $O(n \log n)$   |
| $T(n) = 2T(n/2) + 1$ | to halvdele, næsten intet arbejde | $O(n)$          |

## 3 Sådan løser du (unrolling-metoden)

1. Skriv et par udfoldelser

$$T(n) = T(n/2) + n$$

$$\rightarrow T(n) = T(n/4) + n/2 + n$$

1. Find mønsteret (generelt led)
2. Find, hvornår rekursionen stopper  
(typisk når størrelsen = 1)
3. Summer restleddene
4. Forenkler til Big-O (største led vinder)

## 4 Hvordan hænger algoritmerne sammen?

| Algoritme                     | Rekurrens            | Kompleksitet                    |
|-------------------------------|----------------------|---------------------------------|
| <b>Binær søgning</b>          | $T(n) = T(n/2) + 1$  | <b><math>O(\log n)</math></b>   |
| <b>Mergesort</b>              | $T(n) = 2T(n/2) + n$ | <b><math>O(n \log n)</math></b> |
| <b>Quicksort (gennemsnit)</b> | $T(n) = 2T(n/2) + n$ | <b><math>O(n \log n)</math></b> |
| <b>Quicksort (værste)</b>     | $T(n) = T(n-1) + n$  | <b><math>O(n^2)</math></b>      |
| <b>Lineær rekursion</b>       | $T(n) = T(n-1) + c$  | <b><math>O(n)</math></b>        |

## 5 Mini-guide: Hvad skriver jeg til eksamen?

- ✓ **Opstil rekurrensen** (forklar kort, hvor termene kommer fra)
- ✓ **Udfold 2-3 trin**
- ✓ **Find mønster** og stopbetingelse
- ✓ **Beregn summen** (ofte  $n$ ,  $n \log n$  eller  $n^2$ )
- ✓ **Forenkler til Big-O**

Tip: Hvis du er i tvivl — tegn rekursionstræet og læg arbejdet pr. niveau sammen.

## 6 Hurtige genveje (Master-intuition)

- Halvering  $\rightarrow \log n$  niveauer
- Lineært arbejde på hvert niveau  $\rightarrow n \log n$
- Konstant arbejde på hvert niveau (2 grene)  $\rightarrow n$
- $1+2+\dots+n$  dukker op  $\rightarrow n^2$

## 7 Små regneregler (til forenkling)

- $\sqrt{n} = n^{1/2}$
- $n\sqrt{n} = n^{3/2}$
- $\log(n^k) = k \log n$
- Største led vinder:  $n^2 + n \rightarrow n^2$
- Konstanter ignoreres:  $3n \rightarrow n$

## 8 Tjekliste: Stil disse spørgsmål

- 1 Hvor mange rekursive kald laves?
- 2 Hvor store er delproblemerne?
- 3 Hvad koster arbejdet udenfor rekursionen?
- 4 Hvor dybt bliver rekursionstræet?
- 5 Hvad er summen af arbejdet på alle niveauer?

### Kort konklusion

Rekurrenser er blot **matematikken bag tidskompleksitet** for rekursive algoritmer — især divide-and-conquer.

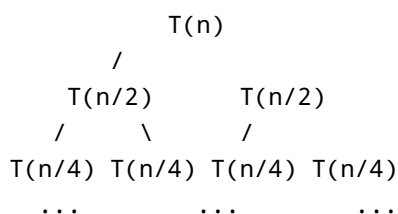
Forstå mønstrene ovenfor, og du kan analysere næsten alle klassiske algoritmer.

*Vil du udvide arket med flere eksempler (fx binær søgning, mergesort med tegning, eller Master-Theorem-skabelon), siger du bare til.*

## 9 Rekursionstræer — mergesort og quicksort

### Mergesort — rekursionstræ

Mergesort deler altid i **to lige store halvdele** og laver derefter et **merge**.



 Arbejde pr. niveau:

- hvert niveau indeholder i alt **n** elementer i merge-arbejde

- der er  $\log_2 n$  niveauer, fordi vi halverer hver gang

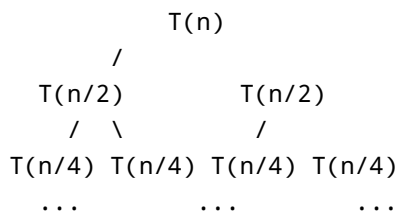
→ Total tid:

$$n + n + n + \dots (\log n \text{ gange}) = n \log n$$

**Konklusion:** Mergesort  $\rightarrow O(n \log n)$

## Quicksort — rekursionstræ (gennemsnit)

Quicksort deler typisk omkring **midt over** i gennemsnit:



● Arbejde pr. niveau:

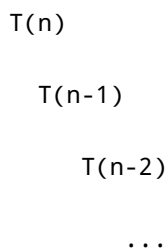
- partitionering koster  $n$  på hvert niveau
- antal niveauer  $\approx \log_2 n$

→ Total tid:  $n \log n$  (samme som mergesort)

**Konklusion:** Quicksort (gennemsnit)  $\rightarrow O(n \log n)$

## Quicksort — værste fald

Hvis pivottet vælges rigtig dårligt (fx altid det mindste/største):



Dette giver summen:

$$1 + 2 + 3 + \dots + n \approx n^2$$

**Konklusion:** Quicksort (værste)  $\rightarrow O(n^2)$

---

### Huskeregul

- Halvering i træets højde  $\rightarrow \log n$  niveauer
  - Hvis hvert niveau koster  $n \rightarrow n \log n$
  - Hvis niveauerne bliver en lineær kæde  $\rightarrow n^2$
- 

*Vil du have, at vi også tegner rekursionstræ for binær søgning og  $T(n)=T(n/2)+n$ ?*